

Fusion problembasierter Skalar-Features mit dem Expression-Graph-GNN

Technischer Bericht: Wiederherstellung des `GlobalEncoder` für
Supervised Learning und Reinforcement Learning

Patrick Fuchs

Bachelorarbeit – GNN/RL Solver-Klassifikation

24. Juni 2026

Abstract

Das Expression-GNN klassifiziert pro Iterationsschritt, welcher numerische Solver (Newton vs. gMGF) schneller konvergiert. Bisher war das Modell ein *rein struktureller* Graph-Encoder: es verarbeitete ausschließlich die one-hot-kodierten Knoten-Features des Ausdrucksgraphen. Die numerischen Problemgrößen (x_0 , $f(x_0)$, $f'(x_0)$, $f''(x_0)$, y_{target}) wurden zwar erhoben, aber nur zur Label-Berechnung genutzt und *nicht* ins Modell gegeben. Dieser Bericht dokumentiert die Wiederherstellung eines dedizierten, kleinen MLP-Pfades (`GlobalEncoder`), der diese Skalare normalisiert, kodiert und *nach* dem Message Passing mit dem Graph-Embedding fusioniert. Das one-hot-Encoding der Struktur bleibt dabei vollständig erhalten; die Skalare bilden einen zweiten, separaten Informationsstrom (*separation of concerns*). Beschrieben werden Datenrepräsentation, Architektur und je ein vollständiger Forward Pass für Supervised Learning (SL) und Reinforcement Learning (RL).

1 Motivation und Kontext

Der Ausdruck eines Wurzelsuchproblems wird vollständig als Graph kodiert: Knoten sind Operatoren/Funktionen/Operanden, Kanten die AST-Struktur. Die Knoten-Features sind kategorial *one-hot* (Knotentyp, Wurzelfarbe, Label) plus ganzzahlige Topologie-Merkmale. Der Zahlenwert einer Konstanten geht dabei verloren – jede Zahl kollabiert auf ein einziges `label_CONSTANT`-One-Hot.

Damit fehlte dem Modell der *Per-Sample-Kontext*: $x_0, f(x_0), f'(x_0), f''(x_0)$ variieren pro Iterationszeile (Auswertungspunkt), nicht nur pro Graph. Zwei Zeilen desselben Graphen mit unterschiedlichen Funktionswerten waren für das Modell ununterscheidbar. Ein früherer `GlobalEncoder`, der genau diese Skalare einband, war im Zuge eines Refactorings entfernt worden (Commit `7e872cd`). Dieser Bericht beschreibt seine originalgetreue Wiederherstellung und Aktivierung für den supervised Klassifikator sowie den Rückbau des RL-Pfades auf denselben Mechanismus.

2 Datenrepräsentation: zwei Informationsströme

Entscheidend ist die Trennung: das One-Hot-Encoding bleibt unverändert, die Skalare kommen *zusätzlich* über einen eigenen Pfad hinzu (Tabelle 1).

Die Skalare werden pro Graph als $[1, k]$ -Tensor angehängt; PyGs Collate stapelt sie über einen Batch zu $[B, k]$ (Batch-Dimension $B = \text{Anzahl Graphen}$).

Strom	Inhalt	Eintrittspunkt
Struktur (one-hot)	data.x , 32 Spalten: node_type (3), root_color (5), label (15), subtree_size/-depth + 4 Histogramm-Bins (6), Anchor-Positional (3)	<i>vorne</i> : Node-Encoder, dann Message Passing
Skalare (kontinuierlich)	data.global_features ; SL: $k=5$ $[x_0, y_{\text{target}}, f, f', f'']$, RL: $k=8$ Solver-State	<i>hinten</i> : kleines MLP, Fusion <i>nach</i> dem Pooling

Table 1: Die beiden getrennten Eingangsströme des Modells.

3 Architektur: GlobalEncoder und Fusion

Sei $X \in \mathbb{R}^{N \times d_{\text{in}}}$ die one-hot-Knotenmatrix (N Knoten im Batch), E die Kantenliste, $\text{batch} \in \{0, \dots, B-1\}^N$ die Knoten-zu-Graph-Zuordnung und $s \in \mathbb{R}^{B \times k}$ die Skalarmatrix. Mit verborgener Breite d und Skalar-Encoder-Breite h_g :

$$\textbf{Node-Encoder: } H^{(0)} = \phi(W_e \text{LN}(X)) \in \mathbb{R}^{N \times d} \quad (1)$$

$$\textbf{Message Passing: } H^{(l)} = \phi_l(\text{GINConv}_l(H^{(l-1)}, E)), \quad l = 1, \dots, L \quad (2)$$

$$\textbf{Pooling: } z = \text{POOL}(H^{(L)}, \text{batch}) \in \mathbb{R}^{B \times d} \quad (3)$$

$$\textbf{Skalar-Encoder: } g = \phi(W_g \text{LN}(s)) \in \mathbb{R}^{B \times h_g} \quad (4)$$

$$\textbf{Fusion: } u = [z \parallel g] \in \mathbb{R}^{B \times (d+h_g)} \quad (5)$$

$$\textbf{Tail: } t = \text{Drop}(\phi(\text{LN}(W_t u))) \in \mathbb{R}^{B \times d} \quad (6)$$

$$\textbf{Head: } \hat{y} = W_h t \in \mathbb{R}^{B \times C} \text{ (SL) bzw. Features} = t \text{ (RL)} \quad (7)$$

Dabei ist ϕ die Aktivierung (Default PReLU), LN LayerNorm, Drop Dropout. Für $h_g=0$ (Skalare deaktiviert) entfällt der Skalar-Encoder, $u = z$, und der Tail-Input ist d – das Modell ist dann identisch zum reinen Struktur-GNN (vollständig rückwärtskompatibel).

Normalisierung. Der Skalar-Encoder normalisiert mit einer lernbaren **LayerNorm**(k) vor dem Linear. Das ist originalgetreu, hat aber eine bewusste Eigenschaft: **LayerNorm** zentriert pro Sample über die k Skalare, d. h. eine *uniforme* additive Verschiebung aller k Werte ist unsichtbar – der Encoder reagiert auf das *relative* Muster, nicht auf das absolute Niveau.

4 Forward Pass: Supervised Learning

Eingang (Batch aus B Graphen, N Knoten gesamt):

Feld	Form	Bedeutung
batch.x	$[N, 32]$	one-hot Knoten-Features (Struktur)
batch.edge_index	$[2, E]$	Kanten
batch.batch	$[N]$	Knoten $\rightarrow$ Graph
batch.global_features	$[B, 5]$	Skalare $[x_0, y_{\text{target}}, f, f', f'']$
batch.y	$[B]$	Label (nur Loss/Metrik, nie Input)

STRUKTUR-PFAD (one-hot)	SKALAR-PFAD (kontinuierlich)
batch.x $[N, 32]$	batch.global_features $[B, 5]$
node_encoder: $\text{LN}(32) \rightarrow \text{Linear}(32, d) \rightarrow \phi$	global_norm: $\text{LN}(5)$
v	global_encoder: $\text{Linear}(5, h_g)$
$[N, d]$	phi
GINConv x L (Message Passing ueber E)	v

```

      v
      [N,d]
      | POOL (mean/add/max) ueber batch.batch
      v
      z [B,d] -----+
                      v
                      | u = concat([z, g], dim=-1) [B, d+h_g] <-- FUSION
                      |
                      | tail: Linear(d+h_g, d)->LN->phi->Dropout
                      v
                      [B, d]
                      | head: Linear(d, C) (C=1 BCE / 2 CE)
                      v
                      logits [B, C]
return (logits, batch.y) -> gewichtete CE/BCE + Metriken (PR-AUC, ...)

```

Bei `use_scalar_features=False` existiert der rechte Pfad nicht ($h_g=0$), und $u = z$.

5 Forward Pass: Reinforcement Learning

Im RL stammt die Eingabe nicht als PyG-Batch, sondern als Gym-Observation-Dict aus dem Mathematica-Environment. Der SB3-Feature-Extractor rekonstruiert daraus erst das PyG-Objekt. Die Skalare ($k=8$ Solver-State) reisen als eigener Observation-Key, *nicht* in `x` einkonkateniert.

Observation-Key	Form	Bedeutung
<code>x</code>	<code>[max_nodes, 32]</code>	one-hot Knoten-Features (nativ)
<code>edge_index</code>	<code>[2, max_edges]</code>	Kanten (gepadding)
<code>global_features</code>	<code>[8]</code>	Solver-State-Skalare
<code>num_nodes, num_edges</code>	<code>[1]</code>	echte Längen (Padding-Schnitt)

Solver-State (Reihenfolge): `currentX`, `yTarget`, `lastStepError`, `fx`, `dfx`, `ddfx`, `kappa`, `lastKappa`. Encoder-Breite $h_g \in \{6, 8\}$ ist ein Optuna-Hyperparameter.

```

Env-Observation ----> CustomGNNFeaturesExtractor.forward
- schneidet Padding ab (num_nodes/num_edges)
- baut Batch.from_data_list(...) inkl. global_features
- ruft ExpressionGNN(x, edge_index, batch, global_features) [classify=False]

STRUKTUR-PFAD                                SKALAR-PFAD
x [N,32]                                     global_features [B,8]
| node_encoder: LN(32)->Linear(32,d)->phi    | global_norm: LN(8)
v                                              | global_encoder: Linear(8,h_g)
GINConv x L -> POOL -> z [B,d] ----+        | phi -> g [B,h_g]
                      v                      v
                      | u = concat([z, g]) [B, d+h_g] <-- FUSION
                      |
                      | tail: Linear(d+h_g, d)->LN->phi->Dropout
                      v
                      Features [B, d] <-- head=None: tail IST das Feature
                      |
                      | -> SB3 Policy/Value (MlpExtractor)
                      v
                      action [B,2] (Solver-Wahl + Toleranz) + value [B,1]

```

Unterschiede SL vs. RL.

	Supervised	Reinforcement
Skalare k	5 (Problemwerte)	8 (Solver-State)
Transport	<code>data.global_features</code> [1, 5]	Gym-Observation-Box [8]
<code>classify</code>	True ($\rightarrow$ head=Linear)	False ($\rightarrow$ head=None)
Ausgang	Logits $[B, C]$	Feature-Vektor $[B, d]$ für PPO

Identisch sind in beiden Fällen der Backbone (ExpressionGNN), die one-hot Struktur-Features und der Fusionsmechanismus.

6 Implementierung

Datei	Änderung
shared/models/gnn_backbones.py	GlobalEncoder wiederhergestellt (global_dim, global_hidden_dim, Fusion, Forward-Signatur)
supervised_learning/preprocessing.py	data.global_features [1, k] in ProblemRunDataset; _parse_scalar (bruchtolerant); Spalten-Guard
supervised_learning/loader_graphgym.py	Config-Flags; global_dim aus Config; global_features in forward
supervised_learning/main.py	Skalar-Auflösung im Custom-Trainingspfad
reinforcement_learning/ (8 Dateien)	Rückbau: Skalare separat statt Broadcast; Observation-Space-Key; global_dim an Call-Sites

Aktivierung (Supervised). Off by default; einschalten per YAML/grid.yaml:

```
expression_graph:
  use_scalar_features: true           # Default: false (reiner Struktur-GNN)
  scalar_features: "x0,y_target,fx,d1x,d2x"
gnn:
  global_hidden_dim: 8                # Breite des kleinen Skalar-MLP (tunebar)
```

Die Curated-CSV muss die Spalten x0, y_target, fx, d1x, d2x enthalten, sonst greift ein expliziter RuntimeError (kein stilles Überspringen).

Kern der Fusion (Backbone).

```
def forward(self, x, edge_index, batch, global_features=None, edge_attr=None):
    x = self.node_encoder(x)
    for i, conv in enumerate(self.convs):
        x = self.layer_activations[i](conv(x, edge_index))
    x_pooled = self.pool_fn(x, batch)
    if self._g_enc_dim > 0:
        if global_features is not None:
            gf = global_features.view(x_pooled.size(0), -1)
            g = self.global_activation(self.global_encoder(self.global_norm(gf)))
        else:
            g = x_pooled.new_zeros(x_pooled.size(0), self._g_enc_dim)
        h = torch.cat([x_pooled, g], dim=-1)
    else:
        h = x_pooled
    out = self.tail(h)
    return self.head(out) if self.head is not None else out
```

7 Verifikation

Die volle Test-Suite (128 Tests) läuft grün. Neue Tests:

- **Backbone:** Fusions-Form ($\text{tail_in} = d + h_g$), Skalar-Sensitivität, Gradient-Fluss in den global_encoder, $h_g=0$ -Invarianz.
- **Supervised:** Skalar-Anhang als $[1, k]$ + korrektes Batch-Collate zu $[B, k]$, Bruch-/Fehlwert-Parsing, Spalten-Guard.

- **RL:** Tests auf den restaurierten Contract (Skalare separat) zurückgesetzt.

Integrativ bestätigt: Flag aus $\Rightarrow$ `global_dim=0`; Flag an $\Rightarrow$ `global_dim=5`, Tail-Input $d + h_g$.

8 Hinweise und Grenzen

- **LayerNorm-Eigenschaft.** `LayerNorm(k)` ist invariant gegenüber einer uniformen Verschiebung aller k Skalare eines Samples. Falls große Funktionswerte später stören, ist der Tausch auf `BatchNorm1d` oder ein vorgeschalteter `asinh`-Transform ein Einzeiler im Encoder.
- **RL-Ripple.** Der Rückbau ändert den Observation-Space-Vertrag; bestehende RL-Checkpoints und Optuna-Studien sind inkompatibel (der Suchraum-Suffix wird automatisch verändert) $\Rightarrow$ frische Studie/Neutraining nötig.
- **Kappa.** `kappa` ist *nicht* Teil des Skalar-Vektors und bleibt strikt hinter dem bestehenden `add_kappa-Gate` (Default `false`).
- **Kein Label-Leak.** `y_target` ist die Lösung (Wurzel) des Problems, nicht das Klassifikationslabel (`faster_algorithm`); es ist ein gültiges Eingangs-Feature für die Solver-Geschwindigkeitsvorhersage.